

BXP Technical Documentation

Developer Reference for the Breathe Exposure Protocol

Version: 2.1

Repository: <https://github.com/bxpprotocol/bxp-spec>

License: Apache 2.0

1. What is BXP?

BXP (Breathe Exposure Protocol) is an open data standard for atmospheric exposure information. It defines:

- * A file format (.bxp.json) for representing air quality measurements in a universal, portable structure
- * A REST API specification that any server implementation must follow
- * A health risk index (BXP_HRI) that converts raw measurements into a single 0-100 health risk score
- * A privacy framework that protects personal exposure data by design
- * A quality control system that labels data reliability consistently across sources

BXP is not a platform, a product, or a company. It is a protocol — like HTTP or PDF. Any developer can implement it; any organisation can run a BXP node.

2. Why It Exists

Air quality sensor data is fragmented. Every manufacturer uses a different format. Government monitoring agencies use different schemas. Research databases are incompatible with each other and with consumer applications.

A developer who wants to build an application using air quality data from multiple cities or multiple sources must write custom parsers for every data source. A researcher who wants to combine data from different monitoring networks must transform everything into a common schema manually.

BXP eliminates this by defining what a .bxp.json file looks like — the same schema for a reading from Accra, Delhi, London, or anywhere else, regardless of what sensor recorded it or what application submitted it.

3. The Problem BXP Addresses

Problem | BXP Solution

Incompatible data formats | Universal .bxp.json schema

Single-pollutant health indices | BXP_HRI – composite, multi-agent health risk index

No privacy standard | SHA-256 hashed IDs, geohash floors,

No open API standard | Fully specified REST API

Platform lock-in | Apache 2.0 license, no proprietary code

4. How the Protocol is Structured

4.1 The Five Stages

Every piece of air quality data flowing through BXP passes through five stages:

LOCATE → DETECT → INTERPRET → PROTECT → REPORT

Stage	What Happens
LOCATE	Geographic context attached (geohash)
DETECT	Data source classified (Tier 1/2/3),
INTERPRET	QC applied, units normalised, quality
PROTECT	BXP_HRI calculated, risk level assigned
REPORT	Reading stored, queryable, shareable

4.2 Data Representation

Every BXP record is a JSON object (*.bpx.json) with a consistent structure:

```
{
  "bpxVersion": "2.0",
  "deviceUuid": "uuid-v4",
  "geohash": "s1v0g",
  "latitude": 5.6037,
  "longitude": -0.1870,
  "timestampUs": 1741342800000000,
  "durationS": 60,
  "indoorOutdoor": "outdoor",
  "agents": [
    { "agentId": "PM2_5", "value": 47.2, "unit": "ug/m3" },
    { "agentId": "NO2", "value": 18.3, "unit": "ppb" }
  ],
  "quality": {
    "flag": "UNVALIDATED",
    "confidence": 0.9,
    "qcMethod": "bpx-sdk-auto"
  },
  "bpxHri": 61.2,
  "bpxHriLevel": "HIGH",
  "payloadHash": "sha256:..."
}
```

Every field has a defined meaning, validated type, and — for required fields — enforced presence.

5. How Data Moves Through the System

```
Data source (sensor / app / manual input)
↓
Python SDK or direct API call
↓
POST /bpx/v2/readings (BXP Reference Server)
```

```

↓
Server validates → computes HRI → stores in SQLite → returns 201
↓
GET /bxp/v2/locations/{geohash}/latest
↓
Application displays data with BXP_HRI and quality flag

```

Any step in this chain can be replaced by a different implementation that follows the same protocol, and the rest of the chain will still work. This is the interoperability guarantee.

6. How Interoperability Works

Interoperability in BXP means: any conforming source can write data that any conforming application can read.

This works because:

1. The schema is defined — agentId values are a closed set from the canonical agent registry
2. Units are canonical — PM2.5 is always $\mu\text{g}/\text{m}^3$, NO2 is always ppb
3. The HRI formula is defined — two implementations computing HRI from the same values must return the same result
4. Quality flags are defined — a VALIDATED flag means the same thing everywhere
5. The API is defined — GET /bxp/v2/readings behaves the same way on any BXP node

A developer building a BXP-compatible application does not need to negotiate data formats with every data source. They implement the BXP schema once.

7. Current Implementation Status

What is fully implemented and working:

Component	Status	Location
.bxp.json format	Complete	SDK: sdk/python/bxp_sdk.py
Reference server	Complete	reference-server/server.py
Python SDK	Complete	sdk/python/bxp_sdk.py
TypeScript SDK	Complete	sdk/typescript/bxp-sdk.ts
CLI tool	Complete	cli/bxp_cli.py
BXP_HRI calculation	Complete	SDK + server
Device registration + auth	Complete	Reference server
Community reports	Complete	Reference server
Privacy (geohash floor, k≥5)	Complete	Reference server
Cryptographic deletion	Complete	Reference server
MQTT bridge	Complete	integrations/mqtt_bridge.py
Sample dataset	Complete	datasets/sample_readings.bxp.json
Docker deployment	Complete	Dockerfile, docker-compose.yml

What is specified but not yet implemented in software:

Component	Status	Notes
Binary .bxp format	Specified	32-byte header design complete
Volume file system	Specified	Server uses SQLite instead
Federated node sync	Specified	/nodes endpoint exists; P2P sync pen
Arduino/ESP32 SDKs	Planned	Target: v2.1
MicroPython SDK	Planned	Target: v2.1

8. Using the Reference Server

Installation

```
cd reference-server
pip install -r requirements.txt
python server.py
```

Server starts at <http://localhost:8000>. Interactive API docs at <http://localhost:8000/docs>.

Optional: AQICN live city data

Set AQICN_TOKEN environment variable to enable live air quality data from 10 default cities. Free token at <https://aqicn.org/api/>. Without this, the server works for submitted readings only.

9. Using the Python SDK

```
from bxp_sdk import write_bxp, read_bxp, validate_bxp, calculate_risk, BXPClient

# Calculate risk from sensor values
risk = calculate_risk(pm25=67.0, no2=31.0, duration="24h", population="sensitive")
print(risk["score"])    # 89.6
print(risk["level"])    # VERY_HIGH
print(risk["advice"])    # "Avoid all outdoor activity."

# Write a .bxp.json file
record = write_bxp("accra_reading.bxp.json", {
    "latitude": 5.6037,
    "longitude": -0.1870,
    "pm25": 47.2,
    "no2": 18.3,
    "temp": 29.0,
})
print(record["bxpHri"])    # 61.2
print(record["bxpHriLevel"]) # HIGH
print(record["payloadHash"]) # sha256:...

# Read and verify a .bxp.json file
data = read_bxp("accra_reading.bxp.json")
print(data["_integrityOk"]) # True – hash verified

# Validate against BXP v2.0 spec
result = validate_bxp("accra_reading.bxp.json")
print(result["valid"])    # True
print(result["summary"])    # VALID BXP v2.0 – 3 agent(s) – HRI 61.2

# Submit to a BXP server
client = BXPClient("http://localhost:8000", device_token="your_token")
result = client.submit(latitude=5.6037, longitude=-0.1870, pm25=47.2)
print(result["readingId"]) # uuid
print(result["bxpHri"])    # 61.2
```

Offline queue (for IoT sensors with intermittent connectivity)

```

from bxp_sdk import OfflineQueue, BXPClient

queue = OfflineQueue("~/bxp/queue.json")
queue.push(latitude=5.6037, longitude=-0.1870, pm25=47.2)

# When connectivity is restored:
client = BXPClient("http://my-bxp-node.example.com")
flushed, failed = queue.flush(client)
print(f"Flushed {flushed} readings, {failed} failed")

```

Async client (for async Python applications)

```

from bxp_sdk import AsyncBXPClient

async with AsyncBXPClient("http://localhost:8000") as client:
    result = await client.submit(latitude=5.6037, longitude=-0.1870, pm25=47.2)
    print(result["bxpHri"])

```

10. Using the CLI

```

# Generate a .bxp.json file from sensor values
python cli/bxp_cli.py generate --pm25 47.2 --no2 18.3 --lat 5.6037 --lon -0.1870

# Read and inspect a .bxp.json file
python cli/bxp_cli.py read reading_20260307_100000.bxp.json

# Validate against BXP v2.0 spec
python cli/bxp_cli.py validate reading_20260307_100000.bxp.json

# Calculate HRI without writing a file
python cli/bxp_cli.py hri --pm25 67.0 --no2 31.0 --duration 8h --population sensitive

# Submit to a server
python cli/bxp_cli.py submit --server http://localhost:8000 --file reading.bxp.json

# Batch submit all .bxp.json files in a directory
python cli/bxp_cli.py batch-submit --dir ./readings/

# Export to CSV or GeoJSON
python cli/bxp_cli.py export reading.bxp.json --format csv

# Generate an HTML map from a folder of readings
python cli/bxp_cli.py map ./readings/ --output map.html

# Configure defaults
python cli/bxp_cli.py config set server http://my-bxp-node.example.com

```

Environment variables: BXP_SERVER_URL, BXP_DEVICE_TOKEN

11. API Examples (curl)

```

# Submit a reading
curl -X POST http://localhost:8000/bxp/v2/readings \

```

```

-H "Content-Type: application/json" \
-H "Authorization: Bearer your_device_token" \
-d '{
  "readings": [{
    "deviceId": "550e8400-e29b-41d4-a716-446655440000",
    "latitude": 5.6037, "longitude": -0.1870,
    "agents": [
      {"agentId": "PM2_5", "value": 47.2, "unit": "ug/m3"},
      {"agentId": "NO2", "value": 18.3, "unit": "ppb"}
    ]
  }]
}'

# Get latest for a location
curl http://localhost:8000/bxp/v2/locations/slv0g/latest

# Get live city data (requires AQICN_TOKEN)
curl http://localhost:8000/bxp/v2/city/accra

# Calculate HRI
curl -X POST "http://localhost:8000/bxp/v2/hri/calculate?duration=24h&population=sensitive" \
-H "Content-Type: application/json" \
-d '{"agentId": "PM2_5", "value": 67.0, "unit": "ug/m3"}'

# Register a device
curl -X POST http://localhost:8000/bxp/v2/devices/register \
-H "Content-Type: application/json" \
-d '{"label": "Rooftop sensor - Accra", "ownerHash": "sha256:..."}'

# Verify a reading's integrity
curl http://localhost:8000/bxp/v2/readings/{reading_id}/verify

# Delete a reading (authenticated)
curl -X DELETE http://localhost:8000/bxp/v2/readings/{reading_id} \
-H "Authorization: Bearer your_device_token"

```

12. The Sample Dataset

datasets/sample_readings.bxp.json contains 10 VALIDATED readings from major world cities: Accra, Lagos, Delhi, Beijing, London, São Paulo, New York, Nairobi, Jakarta, Cairo — spanning all six inhabited continents. All readings use the BXP v2.0 schema and include computed BXP_HRI values.

This dataset demonstrates BXP as a global standard: the same schema represents clean London air (HRI 38.4, MODERATE) and hazardous Delhi air (HRI 100.0, HAZARDOUS) in the same file, queryable with the same code.

13. Limitations

- * The binary .bxp format is fully specified but not yet implemented in software
- * The federated node synchronisation protocol is designed but not yet implemented

- * The reference server is a prototype — it has not been load-tested or security-audited
 - * No third-party implementations of the protocol currently exist
 - * BXP_HRI has not been clinically validated
-

14. Future Development

These are planned, not completed.

- * Binary format encoder/decoder (Python, then C)
 - * Arduino SDK for hardware sensors
 - * ESP32 SDK for IoT deployments
 - * BXP-STREAM real-time data extension
 - * BXP-HEALTH HL7 FHIR R4 mapping
 - * Waterborne and soil contamination extension (v3.0)
-

Copyright 2026 Elvarin — Apache 2.0. The air is public. The data should be too.